

Stage 1: Spatial Vocabulary Acquisition via Visual RL

Complete Mathematical Formalization

Working Document — NeurIPS 2026 Submission

Anonymous

March 11, 2026

Contents

1	Overview and Motivation	2
2	The MDP Formulation	2
2.1	Concept Set	2
2.2	Action Space $\mathcal{A}$	2
2.3	Renderer and State Space	3
2.4	Initial State Distribution ρ_0	3
2.5	Episode Termination	3
3	Policy Architecture: π_θ	3
4	Reward Function	4
4.1	Structure	4
4.2	Visual Accuracy: r_{vis}	4
4.3	Structural Correctness: r_{struct}	4
5	GRPO Training	4
5.1	Why GRPO?	4
5.2	Training Procedure	5
5.3	The KL Term	5
6	Empirical Claims and Experimental Design	5
7	Hyperparameter Summary	6

Overview and Motivation

Large vision-language models (VLMs) possess rich declarative knowledge about visual concepts: they can describe a kidney, a neural circuit, or a metabolic pathway in precise detail. Yet they systematically fail to *express* that knowledge spatially. When asked to draw a kidney, a state-of-the-art VLM produces a generic oval.

We call this the **semantic-spatial gap**: the asymmetry between a model’s declarative knowledge (*knowing that*) and its procedural spatial knowledge (*knowing how to draw*).

Stage 1 of our framework addresses this gap by training a VLM policy to generate accurate spatial programs using a rendering loop as an RL environment—with zero human demonstrations.

Key insight. The Excalidraw rendering loop already provides a meaningful learning signal during inference (render $\rightarrow$ see $\rightarrow$ correct). Stage 1 captures this signal permanently as gradient updates to π_θ , instead of discarding it at session end.

The MDP Formulation

Stage 1 is a finite-horizon, deterministic-transition MDP:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}_{\text{reward}}, \rho_0, T_{\text{max}}) \quad (1)$$

Concept Set

$$\mathcal{C} = \{c_1, c_2, \dots, c_N\} \quad (2)$$

where each $c_i \in \Sigma^*$ is a natural language string describing an educational concept (e.g., “*human kidney cross-section*”, “*citric acid cycle*”). Each episode is conditioned on a single concept $c \sim \text{Uniform}(\mathcal{C})$.

Action Space $\mathcal{A}$

Each action a_t is a **JSON-encoded drawing primitive** from a symbolic vocabulary:

$$\mathcal{A} = \mathcal{A}_{\text{free}} \cup \mathcal{A}_{\text{rect}} \cup \mathcal{A}_{\text{ellipse}} \cup \mathcal{A}_{\text{text}} \cup \mathcal{A}_{\text{arrow}} \cup \{\text{finish}\} \quad (3)$$

The individual primitive sets and their parameters are:

Primitive	Parameters	Domain	Purpose
freedraw	$\mathbf{P} \in \mathbb{R}^{N \times 2}, c \in \mathcal{H}, w > 0$	pts, hex color, width	organic shapes
rect	$x, y, w, h \in \mathbb{R}, c_s, c_f \in \mathcal{H}$	position, size, stroke/fill	rectangular regions
ellipse	$x, y, r_x, r_y \in \mathbb{R}, c_s, c_f \in \mathcal{H}$	center, radii	circular/oval shapes
text	$s \in \Sigma^*, x, y \in \mathbb{R}, f > 0$	string, position, size	labels
arrow	$x_1, y_1, x_2, y_2 \in \mathbb{R}, \ell \in \Sigma^*$	endpoints, label	directed relations
finish	$\emptyset$	—	episode termination

Since π_θ is an autoregressive VLM, action a_t is generated as a token sequence $a_t = (v_1, v_2, \dots, v_L)$ where $v_l \in \mathcal{V}$ (the model vocabulary). The policy factorizes as:

$$\pi_\theta(a_t \mid s_t) = \prod_{l=1}^{|a_t|} \pi_\theta(v_l \mid s_t, v_{1:l-1}) \quad (4)$$

Renderer and State Space

The **renderer** $\mathcal{R}$ is a deterministic, composable function:

$$\mathcal{R} : \mathcal{A}^* \rightarrow \mathbb{R}^{H \times W \times 3} \quad (5)$$

$$\mathcal{R}(a_{1:t}) = \text{compose}(\mathcal{R}(a_{1:t-1}), \text{render}(a_t)), \quad \mathcal{R}(\emptyset) = \mathbf{0}^{H \times W \times 3} \quad (6)$$

The **state** at step t is:

$$s_t = (I_t, c), \quad I_t = \mathcal{R}(a_{1:t}) \in \mathbb{R}^{H \times W \times 3} \quad (7)$$

The **transition function** $\mathcal{T}$ is fully deterministic:

$$s_{t+1} = \mathcal{T}(s_t, a_{t+1}) = (\mathcal{R}(a_{1:t+1}), c) \quad (8)$$

Initial State Distribution ρ_0

$$s_0 = (\mathbf{0}^{H \times W \times 3}, c), \quad c \sim \text{Uniform}(\mathcal{C}) \quad (9)$$

Episode Termination

The episode terminates at step T where:

$$T = \min(\{t : a_t = \text{finish}\} \cup \{T_{\max}\}), \quad T_{\max} = 50 \quad (10)$$

The final canvas is $I_T = \mathcal{R}(a_{1:T})$.

Policy Architecture: π_θ

Critical design choice. π_θ is a *vision-language model*, not a language-only model. At each step it receives the rendered canvas image I_t directly:

$$a_{t+1} \sim \pi_\theta \left(\underbrace{I_t}_{\text{canvas PNG}}, \underbrace{c}_{\text{concept}} \right)$$

An LLM policy receives a text description of the canvas, losing spatial overlap, relative scale, and visual coherence. Our experiments show targeted corrections between rounds are only possible because the model *sees* the canvas.

We instantiate π_θ as **Qwen2-VL-7B**, which encodes the canvas image via a ViT and attends jointly over visual and text tokens:

$$V_{\text{img}} = \text{ViT}(I_t) \in \mathbb{R}^{n_v \times d} \quad (11)$$

$$V_{\text{txt}} = \text{Tokenize}(c) \in \mathbb{R}^{n_t \times d} \quad (12)$$

$$h = \text{TransformerLM}([V_{\text{img}}; V_{\text{txt}}]) \in \mathbb{R}^{(n_v+n_t) \times d} \quad (13)$$

$$\pi_\theta(v_l \mid s_t, v_{1:l-1}) = \text{Softmax}(W_{\text{lm}} h_{-1}) \in \Delta^{|\mathcal{V}|} \quad (14)$$

The full autoregressive factorization from Eq. 4 produces a JSON string token-by-token. Spatial coordinates emerge as digit tokens within the JSON structure.

Reward Function

Structure

Rewards are **terminal only**:

$$r_t = 0 \quad \forall t < T, \quad r_T = \mathcal{R}_{\text{reward}}(I_T, c) \quad (15)$$

The terminal reward decomposes into two orthogonal components:

$$\boxed{\mathcal{R}_{\text{reward}}(I_T, c) = \alpha \cdot r_{\text{vis}}(I_T, c) + \beta \cdot r_{\text{struct}}(a_{1:T}, c)} \quad (16)$$

$$\alpha = 0.7, \quad \beta = 0.3 \quad (\text{validated on held-out concept set}) \quad (17)$$

Visual Accuracy: r_{vis}

$$r_{\text{vis}}(I_T, c) = f_{\phi}(I_T, c) \in [0, 1] \quad (18)$$

where f_{ϕ} is a **frozen VLM judge** with $\phi \neq \theta$ (separate model, preventing circular evaluation). Concretely:

- **Phase 1 (development):** $f_{\phi} = \text{GPT-4o}$, prompted as: “Does this image accurately depict [c]? Score 0–10.” Response divided by 10.
- **Phase 2 (training at scale):** $f_{\phi} = \text{ViT-RoBERTa}$ distilled from 15,000 GPT-4o scored pairs, achieving Spearman $\rho \geq 0.86$ with the oracle while reducing per-step latency by $> 90\%$.

Structural Correctness: r_{struct}

$$r_{\text{struct}}(a_{1:T}, c) = \frac{1}{3} [\mathbf{1}[\text{has_labels}(a_{1:T})] + \mathbf{1}[\text{has_arrows}(a_{1:T})] + \mathbf{1}[\text{in_bounds}(a_{1:T})]] \quad (19)$$

- `has_labels`: at least one `text` primitive in $a_{1:T}$
- `has_arrows`: at least one `arrow` primitive in $a_{1:T}$
- `in_bounds`: all coordinate values within $[0, W] \times [0, H]$

This term is **rule-based**: no model inference required, computed in microseconds per rollout. It provides a signal orthogonal to r_{vis} — enforcing structural completeness independent of visual quality.

GRPO Training

Why GRPO?

Two properties of Stage 1 motivate GRPO over alternatives:

1. **Terminal rewards.** The full drawing (not individual strokes) is evaluated. Critic-based methods (PPO, A2C) require estimating a value function $V(s_t)$ at intermediate steps where the canvas is incomplete. This value estimation is high-variance with no intermediate reward signal. GRPO avoids a critic entirely.
2. **Group-level comparison.** Drawing quality is inherently relative: within the space of all drawings of a kidney, we want to reinforce the better ones. GRPO normalizes rewards within a group of same-concept rollouts, naturally expressing this relative preference.

Training Procedure

For each training step:

Step 1 — Rollout. Sample $G = 8$ independent programs from π_θ for concept c :

$$\mathbf{p}^{(g)} = (a_1^{(g)}, \dots, a_{T^{(g)}}^{(g)}) \sim \pi_\theta(\cdot \mid s_0), \quad g = 1, \dots, G \quad (20)$$

Step 2 — Reward. Compute terminal reward for each rollout:

$$r_g = \mathcal{R}_{\text{reward}}(\mathcal{R}(a_{1:T^{(g)}}^{(g)}), c), \quad g = 1, \dots, G \quad (21)$$

Step 3 — Advantage normalization.

$$\bar{r} = \frac{1}{G} \sum_{g=1}^G r_g, \quad \sigma_r = \sqrt{\frac{1}{G} \sum_{g=1}^G (r_g - \bar{r})^2} \quad (22)$$

$$A_g = \frac{r_g - \bar{r}}{\sigma_r + \epsilon}, \quad \epsilon = 10^{-8} \quad (23)$$

Step 4 — Policy gradient.

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}_{c \sim \mathcal{C}} \left[\frac{1}{G} \sum_{g=1}^G \frac{1}{|\mathbf{p}^{(g)}|} \sum_{t=1}^{T^{(g)}} A_g \cdot \log \pi_\theta(a_t^{(g)} \mid s_{t-1}^{(g)}) \right] + \lambda \cdot \mathbb{D}_{\text{KL}}[\pi_\theta \parallel \pi_{\text{ref}}] \quad (24)$$

Step 5 — Parameter update.

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}_{\text{GRPO}}(\theta) \quad (25)$$

using AdamW with learning rate $\eta = 10^{-5}$ and 100-step linear warmup.

The KL Term

The KL divergence $\mathbb{D}_{\text{KL}}[\pi_\theta \parallel \pi_{\text{ref}}]$ in Eq. 24 anchors θ to the pretrained Qwen2-VL-7B reference weights π_{ref} . This serves two purposes:

- **Prevents catastrophic forgetting** of the model’s pretrained vision and language capabilities.
- **Regularizes exploration:** without it, the policy may exploit degenerate programs that achieve high reward through judge heuristics rather than genuine spatial accuracy.

We set $\lambda = 0.01$ (anneal to zero after Stage 1 convergence).

Empirical Claims and Experimental Design

Claim 1 (Spatial convergence). $\mathbb{E}_{c \sim \mathcal{C}_{\text{val}}} [r_{\text{vis}}(\mathcal{R}(\pi_\theta(c)), c)]$ strictly increases with GRPO training steps over held-out validation concepts.

Test: Plot validation reward curve. Compare to zero-shot VLM baseline (π_{ref}).

Claim 2 (Generalization). A model trained on $\mathcal{C}_{\text{train}}$ (cellular biology) achieves $r_{\text{vis}} > r_{\text{vis}}(\pi_{\text{ref}})$ on $\mathcal{C}_{\text{test}}$ (organic chemistry) without further fine-tuning.

Test: Zero-shot transfer evaluation across 4 concept families.

Claim 3 (Demo efficiency amplifier — bridge to Stage 2). π_{θ}^* (Stage 1 checkpoint) reaches threshold quality in Stage 2 with significantly fewer timestamped demonstrations than π_{ref} directly fine-tuned on Stage 2 data.

Test: Sweep Stage 2 dataset size $\{10, 50, 100, 200 \text{ demos}\}$; compare two curves: (a) $\pi_{\text{ref}} \rightarrow$ Stage 2 fine-tune, (b) $\pi_{\theta}^* \rightarrow$ Stage 2 fine-tune. The crossover is the headline result.

Hyperparameter Summary

Hyperparameter	Value	Note
Model π_{θ}	Qwen2-VL-7B	VLM backbone
Model f_{ϕ} (judge)	GPT-4o / ViT-RoBERTa	Separate from π_{θ}
Group size G	8	Rollouts per concept per step
Max primitives T_{max}	50	Episode length
Reward weights α, β	0.7, 0.3	Validate on held-out set
KL weight λ	0.01	Anneal to 0 at convergence
Learning rate η	10^{-5}	AdamW
Warmup	100 steps	Linear
Canvas size $H \times W$	512×512	Renderer output
Concepts $ \mathcal{C} $	100 (initial)	25 per STEM family